

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего
образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по Лабораторной работе №4

по дисциплине «Практика и Реализация Баз Данных»

Автор: Конанчук Иван Алексеевич

Факультет: ФПИИ

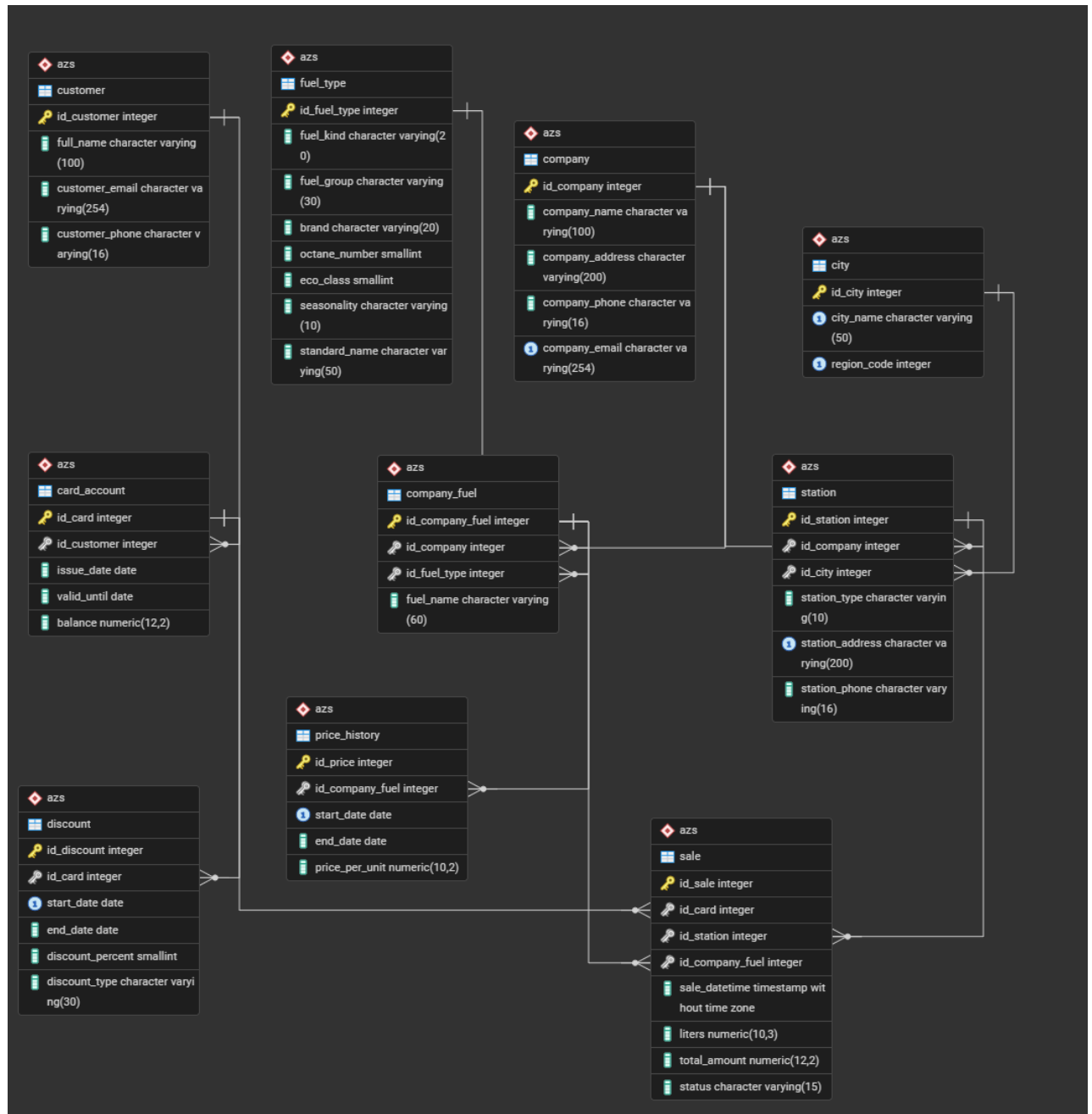
Группа: К3240

ИТМО

Цель работы:

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

1. Схема базы данных



2. Запросы на выборку данных

Запрос 1: Сколько раз заправлял автомобиль каждый клиент за заданный период.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

```

set search_path to azs, public;

with params as (
    select
        date '2026-04-01' as date_from,
        date '2026-04-30' as date_to
)
select
    c.id_customer,
    c.full_name,
    count(s.id_sale) as refuel_count
from customer c
left join card_account ca on c.id_customer = ca.id_customer
left join sale s
    on ca.id_card = s.id_card
    and s.sale_datetime::date between
        (select date_from from params)
    and (select date_to from params)
    and s.status = 'completed'
group by c.id_customer, c.full_name
order by refuel_count desc, c.full_name;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 3   Page No: of 1

⏪ ⏴ ⏵ ⏩

	id_customer [PK] integer 	full_name character varying (100) 	refuel_count bigint 
1	3	Anna Sidorova	0
2	1	Ivan Ivanov	0
3	2	Petr Petrov	0

Запрос 2: Кто из клиентов не приобретал топливо в июле текущего года.

Query

Query History

```

1  set search_path to azs, public;
2
3  select
4      c.id_customer,
5      c.full_name,
6      c.customer_email,
7      c.customer_phone
8  from customer c
9  where not exists (
10     select 1
11     from card_account ca
12     join sale s on ca.id_card = s.id_card
13     where ca.id_customer = c.id_customer
14           and s.status = 'completed'
15           and s.sale_datetime >= date_trunc('year', current_date) + interval '6 months'
16           and s.sale_datetime < date_trunc('year', current_date) + interval '7 months'
17 )
18 order by c.full_name;

```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📤

⬇️

📶

SQL

Showing rows: 1 to 3   Page No: of 1

⏪ ⏩ ⏴ ⏵

	id_customer [PK] integer 	full_name character varying (100) 	customer_email character varying (254) 	customer_phone character varying (16) 
1	3	Anna Sidorova	sidorova@test.ru	+79000000003
2	1	Ivan Ivanov	ivanov@test.ru	+79000000001
3	2	Petr Petrov	petrov@test.ru	+79000000002

Запрос 3: Найти клиента, купившего наибольший объем топлива по всей сети за истекший месяц.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

```
set search_path to azs, public;

select
    c.id_customer,
    c.full_name,
    sum(s.liters) as total_liters
from customer c
join card_account ca on c.id_customer = ca.id_customer
join sale s on ca.id_card = s.id_card
where s.status = 'completed'
    and s.sale_datetime >= timestamp '2026-03-01'
    and s.sale_datetime < timestamp '2026-04-01'
group by c.id_customer, c.full_name
order by total_liters desc
limit 1;
```

Data Output

Messages

Notifications

≡+

SQL

Showing rows: 1 to 1 Page No: 1 of 1

	id_customer [PK] integer	full_name character varying (100)	totalLiters numeric
1	1	Ivan Ivanov	45.750

Запрос 4: Вывести данные клиента, купившего топлива на наибольшую сумму в заданный день.

QueryQuery History

```

1  set search_path to azs, public;
2
3  with params as (
4      select date '2026-03-07' as target_date
5  )
6  select
7      c.id_customer,
8      c.full_name,
9      c.customer_email,
10     c.customer_phone,
11     sum(s.total_amount) as day_total_amount
12 from customer c
13 join card_account ca on c.id_customer = ca.id_customer
14 join sale s on ca.id_card = s.id_card
15 where s.status = 'completed'
16     and s.sale_datetime::date = (select target_date from params)
17 group by c.id_customer, c.full_name, c.customer_email, c.customer_phone
18 order by day_total_amount desc
19 limit 1;

```

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑️

🔄

⬇️

📈

SQL

Showing rows: 1 to 1

✎

📄

 Page No: 1 of 1

⏪

⏴

⏵

⏩

	id_customer [PK] integer	full_name character varying (100)	customer_email character varying (254)	customer_phone character varying (16)	day_total_amount numeric
1	3	Anna Sidorova	sidorova@test.ru	+79000000003	2339.16

Запрос 5: Какое топливо пользовалось наибольшим спросом в прошедшем году на АЗС конкретного поставщика.

Query

Query History

1

set search_path to azs, public;

2

3

with params as (

4

select 'Lukoil'::varchar as supplier_name

5

)

6

select

7

co.company_name,

8

cf.id_company_fuel,

9

cf.fuel_name,

10

ft.fuel_kind,

11

ft.brand,

12

sum(s.liters) as total_liters,

13

count(s.id_sale) as sales_count

14

from sale s

15

join station st on s.id_station = st.id_station

16

join company co on st.id_company = co.id_company

17

join company_fuel cf on s.id_company_fuel = cf.id_company_fuel

18

join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type

19

where s.status = 'completed'

20

and co.company_name = (select supplier_name from params)

21

and s.sale_datetime >= current_date - interval '1 year'

22

and s.sale_datetime < current_date + interval '1 day'

23

group by

24

co.company_name,

25

cf.id_company_fuel,

26

cf.fuel_name,

27

ft.fuel_kind,

28

ft.brand

29

order by total_liters desc

30

limit 1;

Data Output

Messages

Notifications

≡

+

📄

⌵

🗑️

⌵

🔍

⌵

⬇️

⌵

SQL

Showing

	company_name character varying (100)	id_company_fuel integer	fuel_name character varying (60)	fuel_kind character varying (20)	brand character varying (20)	total_liters numeric	sales_count bigint
1	Lukoil	1	AI-92 Lukoil	gasoline	AI-92	30.500	1

Запрос 6: Сколько топлива каждого вида было продано за прошедший месяц по каждому поставщику на каждой заправке.

Query

Query History

```
1  set search_path to azs, public;
2
3  select
4      co.company_name,
5      st.id_station,
6      st.station_address,
7      ft.fuel_kind,
8      ft.brand,
9      cf.fuel_name,
10     sum(s.liters) as total_liters,
11     count(s.id_sale) as sales_count
12 from sale s
13 join station st on s.id_station = st.id_station
14 join company co on st.id_company = co.id_company
15 join company_fuel cf on s.id_company_fuel = cf.id_company_fuel
16 join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
17 where s.status = 'completed'
18       and s.sale_datetime >= timestamp '2026-03-01'
19       and s.sale_datetime < timestamp '2026-04-01'
20 group by
21     co.company_name,
22     st.id_station,
23     st.station_address,
24     ft.fuel_kind,
25     ft.brand,
26     cf.fuel_name
27 order by
28     co.company_name,
29     st.id_station,
30     total_liters desc;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 4

	company_name character varying (100)	id_station integer	station_address character varying (200)	fuel_kind character varying (20)	brand character varying (20)	fuel_name character varying (60)	total_liters numeric	sales_count bigint
1	Gazprom Neft	3	Saint Petersburg, Moskovsky Ave, ...	gasoline	AI-95	AI-95 G-Drive	15.250	1
2	Gazprom Neft	4	Kazan, Pobedy Ave, 40	diesel	DT	DT Gazprom Neft	35.000	1
3	Lukoil	1	Saint Petersburg, Nevsky Ave, 10	gasoline	AI-92	AI-92 Lukoil	30.500	1
4	Lukoil	2	Moscow, Leningradsky Ave, 15	gasoline	AI-95	AI-95 Lukoil	20.000	1

Запрос 7: Какая из заправок продала топлива на наибольшую сумму по всем автозаправкам за последний год.

Query

Query History

```

1  set search_path to azs, public;
2
3  select
4      st.id_station,
5      co.company_name,
6      ci.city_name,
7      st.station_address,
8      st.station_type,
9      sum(s.total_amount) as total_revenue,
10     sum(s.liters) as total_liters,
11     count(s.id_sale) as sales_count
12 from sale s
13 join station st on s.id_station = st.id_station
14 join company co on st.id_company = co.id_company
15 join city ci on st.id_city = ci.id_city
16 where s.status = 'completed'
17       and s.sale_datetime >= current_date - interval '1 year'
18       and s.sale_datetime < current_date + interval '1 day'
19 group by
20     st.id_station,
21     co.company_name,
22     ci.city_name,
23     st.station_address,
24     st.station_type
25 order by total_revenue desc
26 limit 1;

```

Data Output

Messages

Notifications

≡+

📄

▼

📁

▼

🗑️

🔄

⬇️

📶

SQL

Showing rows:

	id_station integer	company_name character varying (100)	city_name character varying (50)	station_address character varying (200)	station_type character varying (10)	total_revenue numeric	total_liters numeric	sales_count bigint
1	4	Gazprom Neft	Kazan	Kazan, Pobedy Ave, 40	AZGS	2339.16	35.000	1

3. Представления

Представление 1: Все АЗС и все виды топлива, которые они продают.

Создаем:

Query

Query History

```

1  set search_path to azs, public;
2
3  create or replace view v_station_fuel_list as
4  select
5      st.id_station,
6      co.company_name,
7      ci.city_name,
8      st.station_type,
9      st.station_address,
10     st.station_phone,
11     cf.id_company_fuel,
12     cf.fuel_name,
13     ft.fuel_kind,
14     ft.fuel_group,
15     ft.brand,
16     ft.octane_number,
17     ft.eco_class,
18     ft.seasonality,
19     ft.standard_name
20 from station st
21 join company co on st.id_company = co.id_company
22 join city ci on st.id_city = ci.id_city
23 join company_fuel cf on co.id_company = cf.id_company
24 join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type;

```

Data Output

Messages

Notifications

CREATE VIEW

Query returned successfully in 128 msec.

Проверяем:

Query

Query History

1

select *

2

from v_station_fuel_list

3

order by company_name, id_station, brand;

Data Output

Messages

Notifications

<

octane_number smallint	eco_class smallint	seasonality character varying (10)	standard_name character varying (50)
95	5	[null]	Euro-5
[null]	5	Z	Euro-5
95	5	[null]	Euro-5
[null]	5	Z	Euro-5
92	5	[null]	Euro-5
95	5	[null]	Euro-5
92	5	[null]	Euro-5
95	5	[null]	Euro-5

Представление 2: Самая прибыльная АЗГС за истекший месяц для каждого производителя.

Создаем:

```
set search_path to azs, public;
```

```
create or replace view v_best_azgs_by_company_last_month as
with period_bounds as (
    select
        date_trunc('month', max(sale_datetime)) as date_from,
        date_trunc('month', max(sale_datetime)) + interval '1 month' as date_to
    from sale
    where status = 'completed'
),
station_revenue as (
    select
        co.id_company,
        co.company_name,
        st.id_station,
        ci.city_name,
        st.station_address,
        st.station_type,
        sum(s.total_amount) as total_revenue,
        sum(s.liters) as total_liters,
        count(s.id_sale) as sales_count
    from sale s
    join station st on s.id_station = st.id_station
    join company co on st.id_company = co.id_company
    join city ci on st.id_city = ci.id_city
    cross join period_bounds pb
    where s.status = 'completed'
    and st.station_type = 'AZGS'
    and s.sale_datetime >= pb.date_from
    and s.sale_datetime < pb.date_to
```

```

group by
    co.id_company,
    co.company_name,
    st.id_station,
    ci.city_name,
    st.station_address,
    st.station_type
),
ranked_stations as (
    select
        station_revenue.*,
        row_number() over (
            partition by id_company
            order by total_revenue desc
        ) as rn
    from station_revenue
)
select
    id_company,
    company_name,
    id_station,
    city_name,
    station_address,
    station_type,
    total_revenue,
    total_liters,
    sales_count
from ranked_stations
where rn = 1;

```

Проверяем:

Query		Query History	
1	select *		
2	from v_best_azgs_by_company_last_month		
3	order by total_revenue desc;		
Data Output			
Messages			
Notifications			
SQL			
	id_company	company_name	id_station
	integer	character varying (100)	integer
1	2	Gazprom Neft	4
	city_name	station_address	station_type
	character varying (50)	character varying (200)	character varying (10)
	Kazan	Kazan, Pobedy Ave, 40	AZGS
	total_revenue	total_liters	sales_count
	numeric	numeric	bigint
	2339.16	35.000	1

4. Модификация данных с подзапросами

INSERT:

Добавим новую продажу для клиента Ivan Ivanov, топливо AI-95 Lukoil, станция Lukoil в Санкт-Петербурге. ID и связи берутся через подзапросы.

До:

Query

Query History

1

set search_path to azs, public;

2

3

select *

4

from sale

5

order by id_sale;

Data Output

Messages

Notifications

SQL

	id_sale [PK] integer	id_card integer	id_station integer	id_company_fuel integer	sale_datetime timestamp without time zone	liters numeric (10,3)	total_amount numeric (12,2)	status character varying (15)
1		1	1	1	2026-03-05 10:15:00	30.500	1677.65	completed
2		2	2	2	2026-03-06 12:40:00	20.000	1142.04	completed
3		3	3	4	2026-03-07 09:20:00	35.000	2339.16	completed
4		4	1	3	2026-03-08 18:05:00	15.250	915.61	completed

Запрос:

```
set search_path to azs, public;
```

```
insert into sale (
```

```
    id_sale,
```

```
    id_card,
```

```
    id_station,
```

```
    id_company_fuel,
```

```
    sale_datetime,
```

```
    liters,
```

```
    total_amount,
```

```
    status
```

```
)
```

```
select
```

```
    (select coalesce(max(id_sale), 0) + 1 from sale) as id_sale,
```

```
    (
```

```
        select ca.id_card
```

```
        from card_account ca
```

```
        join customer c on ca.id_customer = c.id_customer
```

```
        where c.full_name = 'Ivan Ivanov'
```

```
        limit 1
```

```
    ) as id_card,
```

```
    (
```

```
        select st.id_station
```

```
        from station st
```

```
        join company co on st.id_company = co.id_company
```

```
        join city ci on st.id_city = ci.id_city
```

```
        where co.company_name = 'Lukoil'
```

```

        and ci.city_name = 'Saint Petersburg'
    limit 1
) as id_station,
(
    select cf.id_company_fuel
    from company_fuel cf
    join company co on cf.id_company = co.id_company
    join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
    where co.company_name = 'Lukoil'
        and ft.brand = 'AI-95'
    limit 1
) as id_company_fuel,
now() as sale_datetime,
12.500 as liters,
round(
    12.500 * (
        select ph.price_per_unit
        from price_history ph
        join company_fuel cf on ph.id_company_fuel = cf.id_company_fuel
        join company co on cf.id_company = co.id_company
        join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
        where co.company_name = 'Lukoil'
            and ft.brand = 'AI-95'
            and current_date between ph.start_date and coalesce(ph.end_date, date
'9999-12-31')
        limit 1
    ) * 0.95,
    2
) as total_amount,
'processing' as status;

```

Проверяем

Query

Query History

1

select *

2

from sale

3

order by id_sale;

Data Output

Messages

Notifications

</

UPDATE

Увеличим актуальную цену топлива AI-95 Lukoil на 2%.

До:

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

```
set search_path to azs, public;

select
    ph.id_price,
    co.company_name,
    ft.brand,
    cf.fuel_name,
    ph.price_per_unit,
    ph.start_date,
    ph.end_date
from price_history ph
join company_fuel cf on ph.id_company_fuel = cf.id_company_fuel
join company co on cf.id_company = co.id_company
join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
where co.company_name = 'Lukoil'
    and ft.brand = 'AI-95';
```

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	id_price integer	company_name character varying (100)	brand character varying (20)	fuel_name character varying (60)	price_per_unit numeric (10,2)	start_date date	end_date date
1	3	Lukoil	AI-95	AI-95 Lukoil	61.40	2026-03-01	[null]

Запрос:

Query	Query History
1	<code>set search_path to azs, public;</code>
2	
3	<code>update price_history</code>
4	<code>set price_per_unit = round(price_per_unit * 1.02, 2)</code>
5	<code>where id_company_fuel in (</code>
6	<code> select cf.id_company_fuel</code>
7	<code> from company_fuel cf</code>
8	<code> join company co on cf.id_company = co.id_company</code>
9	<code> join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type</code>
10	<code> where co.company_name = 'Lukoil'</code>
11	<code> and ft.brand = 'AI-95'</code>
12	<code>)</code>
13	<code>and current_date between start_date and coalesce(end_date, date '9999-12-31');</code>

После:

Query Query History

```

1 select
2     ph.id_price,
3     co.company_name,
4     ft.brand,
5     cf.fuel_name,
6     ph.price_per_unit,
7     ph.start_date,
8     ph.end_date
9 from price_history ph
10 join company_fuel cf on ph.id_company_fuel = cf.id_company_fuel
11 join company co on cf.id_company = co.id_company
12 join fuel_type ft on cf.id_fuel_type = ft.id_fuel_type
13 where co.company_name = 'Lukoil'
14        and ft.brand = 'AI-95';

```

Data Output Messages Notifications

	id_price integer	company_name character varying (100)	brand character varying (20)	fuel_name character varying (60)	price_per_unit numeric (10,2)	start_date date	end_date date
1	3	Lukoil	AI-95	AI-95 Lukoil	62.63	2026-03-01	[null]

DELETE

Удалить незавершенные продажи клиента Ivan Ivanov. Я специально вставил продажу со статусом processing, теперь ее удалю.

До:

Query Query History

```

1 set search_path to azs, public;
2
3 select
4     s.id_sale,
5     c.full_name,
6     s.sale_datetime,
7     s.total_amount,
8     s.status
9 from sale s
10 join card_account ca on s.id_card = ca.id_card
11 join customer c on ca.id_customer = c.id_customer
12 where c.full_name = 'Ivan Ivanov'
13 order by s.id_sale;

```

Data Output Messages Notifications

	id_sale integer	full_name character varying (100)	sale_datetime timestamp without time zone	total_amount numeric (12,2)	status character varying (15)
1	1	Ivan Ivanov	2026-03-05 10:15:00	1677.65	completed
2	4	Ivan Ivanov	2026-03-08 18:05:00	915.61	completed
3	5	Ivan Ivanov	2026-05-30 00:48:36.57194	729.13	processing

Запрос:

```
Query  Query History
1      set search_path to azs, public;
2
3      delete from sale
4      where id_card in (
5          select ca.id_card
6          from card_account ca
7          join customer c on ca.id_customer = c.id_customer
8          where c.full_name = 'Ivan Ivanov'
9      )
10     and status = 'processing';
```

После:

Query Query History

```
1      select
2          s.id_sale,
3          c.full_name,
4          s.sale_datetime,
5          s.total_amount,
6          s.status
7      from sale s
8      join card_account ca on s.id_card = ca.id_card
9      join customer c on ca.id_customer = c.id_customer
10     where c.full_name = 'Ivan Ivanov'
11     order by s.id_sale;
```

Data Output Messages Notifications

≡+

📄

▼

🗑️

📄

📄

📄

📄

SQL

Showing rows: 1 to 2

✎

📄

 Page No:

1

 of 1

⏪

⏴

⏵

⏩

	id_sale integer 🔒	full_name character varying (100) 🔒	sale_datetime timestamp without time zone 🔒	total_amount numeric (12,2) 🔒	status character varying (15) 🔒
1	1	Ivan Ivanov	2026-03-05 10:15:00	1677.65	completed
2	4	Ivan Ivanov	2026-03-08 18:05:00	915.61	completed

Транзакция удалена!

5. Графическое представление запроса

Возьмем запрос про максимальную выручку, нажимаем Explain Analyze:

Query

Query History

Show queries generated internally by pgAdmin?

Remove

Remove All

Today - 5/30/2026

▶ create schema if not ex...

01:03:28

▶ select s.id_sale, s.sa...

01:00:27

▶ select s.id_sale, s.sa...

01:00:25

▶ select s.id_sale, s.sa...

01:00:24

▶ select s.id_sale, s.sa...

01:00:19

▶ select s.id_sale, s.sa...

01:00:14

▶ select s.id_sale, s.sa...

00:58:15

▶ set search_path to azs,...

00:57:14

▶ select s.id_sale, c.ful...

00:55:39

▶ set search_path to azs,...

00:55:17

▶ set search path to azs,...

5/30/2026 1:00:27 AM

72 msec

1

Date

Rows affected

Duration

Copy

Copy to Query Editor

```

select
    s.id_sale,
    s.sale_datetime,
    c.full_name,
    cf.fuel_name,
    s.total_amount
from sale s
join card_account ca on s.i
join customer c on ca.id_cu
join company_fuel cf on s.i
where s.status = 'completed

```

Messages

Successfully run. Total query runtime: 72 msec. 1 rows affected.

В ней отображается история команд текущего сеанса, время выполнения и количество обработанных строк.

8. Индексы и сравнение времени

Сейчас тестовых данных у меня мало, поэтому для того чтобы показать работу индексов сгенерируем дополнительных данных:

Query Query History

```
1 set search_path to azs, public;
2
3 insert into sale (
4     id_sale,
5     id_card,
6     id_station,
7     id_company_fuel,
8     sale_datetime,
9     liters,
10    total_amount,
11    status
12 )
13 select
14     1000 + g as id_sale,
15     ((g - 1) % 3) + 1 as id_card,
16     ((g - 1) % 4) + 1 as id_station,
17     ((g - 1) % 4) + 1 as id_company_fuel,
18     timestamp '2026-04-01 00:00:00' + (g || ' minutes')::interval as sale_datetime,
19     20.000 as liters,
20     1200.00 as total_amount,
21     'completed' as status
22 from generate_series(1, 20000) as g
23 where not exists (
24     select 1
25     from sale s
26     where s.id_sale = 1000 + g
27 );
```

Data Output Messages Notifications

INSERT 0 20000

Query returned successfully in 671 msec.

Query Query History

```
1 select count(*) as sale_count
2 from sale;
```

Data Output Messages Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 1

🔍

📺

 Page No: 1

	sale_count bigint 🔒
1	20004

Данные заполнились.

Запрос без индекса:

Query

Query History

1

`explain analyze`
`select`
`id_sale,`
`sale_datetime,`
`liters,`
`total_amount`
`from sale`
`where sale_datetime >= timestamp '2026-04-10 00:00:00'`
`and sale_datetime < timestamp '2026-04-11 00:00:00';`

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗑️

⬇️

⚡

SQL

Showing rows: 1 to 8

✎

📧

Page No: 1 of 1

⏪

⏴

⏵

⏩

QUERY PLAN

text

1

Seq Scan on sale (cost=0.00..487.06 rows=1439 width=22) (actual time=0.852..1.483 rows=1439)

2

Filter: ((sale_datetime >= '2026-04-10 00:00:00'::timestamp without time zone) AND (sale_datetime < '2026-04-11 00:00:00'::timestamp without time zone))

3

Rows Removed by Filter: 18564

4

Buffers: shared hit=187

5

Planning:

6

Buffers: shared hit=17

7

Planning Time: 0.107 ms

8

Execution Time: 1.533 ms

Время - 1.533 мс.

Создаем простой индекс:

Query

Query History

1

`create index idx_sale_datetime`
`on sale (sale_datetime);`

`analyze sale;`

Запрос с индексом:

QueryQuery History

1

explain analyze
select
id_sale,
sale_datetime,
liters,
total_amount
from sale
where sale_datetime >= timestamp '2026-04-10 00:00:00'
and sale_datetime < timestamp '2026-04-11 00:00:00';

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

⤴

SQL

Showing rows: 1 to 8

✎

🖨

Page No:

1

of 1

⏪

⏴

⏵

⏩

	QUERY PLAN text
1	Index Scan using idx_sale_datetime on sale (cost=0.29..66.07 rows=1439 width=22) (actual tim
2	Index Cond: ((sale_datetime >= '2026-04-10 00:00:00'::timestamp without time zone) AND (sale
3	Index Searches: 1
4	Buffers: shared hit=14 read=6
5	Planning:
6	Buffers: shared hit=27 read=1
7	Planning Time: 1.352 ms
8	Execution Time: 0.354 ms

Время - 0.354 мс уменьшилось почти в 5 раз.

Теперь проверим запрос сразу по станции и дате:

Query

Query History

1

`explain analyze`
`select`
`id_sale,`
`id_station,`
`sale_datetime,`
`total_amount`
`from sale`
`where id_station = 2`
`and sale_datetime >= timestamp '2026-04-10 00:00:00'`
`and sale_datetime < timestamp '2026-04-20 00:00:00'`
`order by sale_datetime;`

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

📦

⬇️

⤴️

SQL

Showing rows: 1 to 10

✎

🖨️

Page No: 1 of 1

⏪ ⏩ ⏴ ⏵

	QUERY PLAN
	text
1	Index Scan using idx_sale_datetime on sale (cost=0.29..311.69 rows=1760 width=21) (actual time
2	Index Cond: ((sale_datetime >= '2026-04-10 00:00:00'::timestamp without time zone) AND (sale_d
3	Filter: (id_station = 2)
4	Rows Removed by Filter: 5281
5	Index Searches: 1
6	Buffers: shared hit=73 read=14
7	Planning:
8	Buffers: shared hit=25 read=1
9	Planning Time: 0.206 ms
10	Execution Time: 1.256 ms

Время - 1.256 мс.

Создаем составной индекс:

```
Query  Query History
1  create index idx_sale_station_datetime
2  on sale (id_station, sale_datetime);
3
4  analyze sale;

Data Output  Messages  Notifications
ANALYZE

Query returned successfully in 80 msec.
```

После проверки получилось такое же время, поэтому добавим еще продаж:

```
Query  Query History
1  set search_path to azs, public;
2
3  insert into sale (
4      id_sale,
5      id_card,
6      id_station,
7      id_company_fuel,
8      sale_datetime,
9      liters,
10     total_amount,
11     status
12 )
13 select
14     (select coalesce(max(id_sale), 0) from sale) + g as id_sale,
15     ((g - 1) % 3) + 1 as id_card,
16     ((g - 1) % 4) + 1 as id_station,
17     ((g - 1) % 4) + 1 as id_company_fuel,
18     timestamp '2026-04-01 00:00:00' + (g || ' minutes')::interval as sale_datetime,
19     20.000 as liters,
20     1200.00 as total_amount,
21     'completed' as status
22 from generate_series(1, 300000) as g;

Data Output  Messages  Notifications
ERROR:  new row for relation "sale" violates check constraint "ck_sale_datetime"
Failing row contains (106042, 1, 2, 2, 2026-05-30 01:22:00, 20.000, 1200.00, completed).

SQL state: 23514
Detail: Failing row contains (106042, 1, 2, 2, 2026-05-30 01:22:00, 20.000, 1200.00, completed).
```

Выдало ошибку, что закономерно, я не учел, что 300000 минут от 2026-04-01 дойдет дальше нынешней даты, и PostgreSQL не пустил её.

Создадим запросы в 2025 году:

Query	Query History
1	<code>set search_path to azs, public;</code>
2	
3	<code>insert into sale (</code>
4	<code> id_sale,</code>
5	<code> id_card,</code>
6	<code> id_station,</code>
7	<code> id_company_fuel,</code>
8	<code> sale_datetime,</code>
9	<code> liters,</code>
10	<code> total_amount,</code>
11	<code> status</code>
12	<code>)</code>
13	<code>select</code>
14	<code> (select coalesce(max(id_sale), 0) from sale) +</code>
15	<code> ((g - 1) % 3) + 1 as id_card,</code>
16	<code> ((g - 1) % 4) + 1 as id_station,</code>
17	<code> ((g - 1) % 4) + 1 as id_company_fuel,</code>
18	<code> timestamp '2025-01-01 00:00:00' + (g ' minut</code>
19	<code> 20.000 as liters,</code>
20	<code> 1200.00 as total_amount,</code>
21	<code> 'completed' as status</code>
22	<code>from generate_series(1, 300000) as g;</code>

Data Output	Messages	Notifications
<code>INSERT 0 300000</code>		
<code>Query returned successfully in 8 secs 434 msec.</code>		

Тестируем без индекса:

Query

Query History

```

1  set search_path to azs, public;
2
3  drop index if exists idx_sale_station_datetime;
4  drop index if exists idx_sale_datetime;
5
6  analyze sale;
7
8  explain analyze
9  select
10     id_sale,
11     id_station,
12     sale_datetime,
13     total_amount
14 from sale
15 where id_station = 2
16     and sale_datetime >= timestamp '2025-04-10 10:00:00'
17     and sale_datetime < timestamp '2025-04-10 11:00:00'
18 order by sale_datetime;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

Showing rows: 1 to 16

✎

📧

Page No: 1 of 1

⏪

⏴

⏵

⏩

	QUERY PLAN text
9	-> Parallel Seq Scan on sale (cost=0.00..6285.16 rows=10 width=21) (actual time=6.293..11.803 row
10	Filter: ((sale_datetime >= '2025-04-10 10:00:00'::timestamp without time zone) AND (sale_datetim
11	Rows Removed by Filter: 159994
12	Buffers: shared hit=2991
13	Planning:
14	Buffers: shared hit=15
15	Planning Time: 0.121 ms
16	Execution Time: 43.575 ms

И с тем же составным индексом:

Query

Query History

1

```
explain analyze
select
  id_sale,
  id_station,
  sale_datetime,
  total_amount
from sale
where id_station = 2
      and sale_datetime >= timestamp '2025-04-10 10:00:00'
      and sale_datetime < timestamp '2025-04-10 11:00:00'
order by sale_datetime;
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows: 1 to 8

✎

🖨

 Page No:

1

 of 1

⏪

⏴

⏵

⏩

QUERY PLAN

text

1

Index Scan using idx_sale_station_datetime on sale (cost=0.42..58.75 rows=14 width=21) (actual rows=8)

2

Index Cond: ((id_station = 2) AND (sale_datetime >= '2025-04-10 10:00:00'::timestamp without time zone AND sale_datetime < '2025-04-10 11:00:00'::timestamp without time zone))

3

Index Searches: 1

4

Buffers: shared hit=2 read=3

5

Planning:

6

Buffers: shared hit=30 read=1

7

Planning Time: 1.326 ms

8

Execution Time: 0.076 ms

✓

Successfully run. Total query runtime: 61 msec. 8 rows returned.

Теперь видно разницу.

Таблица сравнения:

Запрос	Индекс	Время без индекса	Время с индексом
Поиск продаж по дате	idx_sale_datetime	1.533 ms	0.354 ms
Поиск продаж по станции и дате	idx_sale_station_datetime	43.575 ms	0.076 ms

Удалим индексы:

```

Query  Query History
1      set search_path to azs, public;
2
3      drop index if exists idx_sale_datetime;
4      drop index if exists idx_sale_station_datetime;

```

Сделаем бекап:

Backup (Database: azs_sales_db)

General
Data Options
Query Options
Table Options
Options
Objects

Filename
lab4

Format
Plain

Compression ratio

Encoding
Select an item...

Number of jobs

Role name
Select an item...

Вывод:

В ходе лабораторной работы были выполнены запросы на выборку данных из базы azs_sales_db по индивидуальному варианту "Анализ продаж автомобильного топлива в сети АЗС". Были составлены запросы для анализа активности клиентов, поиска клиентов без покупок в заданный период, определения клиентов с максимальным объемом и суммой покупок, а также для анализа спроса на топливо и выручки по заправкам.

Также были выполнены запросы на модификацию данных INSERT, UPDATE и DELETE с использованием подзапросов. Для анализа производительности были построены планы выполнения запросов через EXPLAIN ANALYZE, созданы простой и составной индексы, после чего время выполнения запросов было сравнено до и после создания индексов. После проверки индексы были удалены. В результате были закреплены навыки работы с Query Tool, представлениями, планами выполнения запросов и индексами в PostgreSQL.